Claude Code in Action

Tool Use · Context · Thinking · Hooks · MCP · SDK

mostly Opus 4.6

Course Overview

What is Claude Code?

- Coding assistants
- Claude Code vs. Copilot
- Coding LLM benchmarks

Getting Hands On

- Setup, context, permissions
- Making changes
- Thinking & effort levels
- Commands & shortcuts
- MCP servers (+ Context7)
- GitHub integration

Hooks, SDK & Tips

- Hook architecture
- PreToolUse / PostToolUse
- Implementing hooks
- Claude Agent SDK
- Workflow tips & pitfalls

What Is a Coding Assistant?

A coding assistant uses language models to tackle complex programming tasks.

How it works (same as a human developer):

1. **Gather context** — read files, understand errors, find relevant code
2. **Formulate a plan** — decide on a fix or implementation strategy
3. **Take action** — edit files, run commands, verify results

Steps 1 and 3 require the model to *interact with the outside world*.
But language models can only process text and return text...

Claude Code vs. GitHub Copilot

Claude Code

- **Agentic orchestration**: plans and executes entire tasks autonomously
- 1M token context window — full repo understanding
- 300+ MCP integrations (Slack, Sentry, DBs, . . .)
- Git-native: stages, commits, creates PRs
- Best for **delegating entire tasks**

GitHub Copilot

- **Multi-model wrapper**: GPT, Claude, Gemini behind one UI
- Developer-directed: you guide each step
- Deep GitHub ecosystem (Actions, security, org controls)
- Agent mode for multi-file, but iterative
- Best for **speeding up known work**

Why orchestration matters (SWE-bench)

Same Claude 3.5 Sonnet: **49%** basic scaffold → **64%** agentic scaffolding (+30% relative gain).

Orchestration intelligence matters as much as raw model intelligence.

Top LLMs for Coding (March 2026)

SWE-bench Verified (500 real GitHub issues)

#	Model	Score
1	Claude Opus 4.5 (Anthropic)	80.9%
2	Claude Opus 4.6 (Anthropic)	80.8%
3	Gemini 3.1 Pro (Google)	80.6%
4	MiniMax M2.5 (open-weight)	80.2%
5	GPT-5.2 (OpenAI)	80.0%
6	Claude Sonnet 4.6 (Anthropic)	79.6%
7	GLM-5 (Zhipu AI)	77.8%

SWE-bench Pro (1,865 multi-lang tasks)

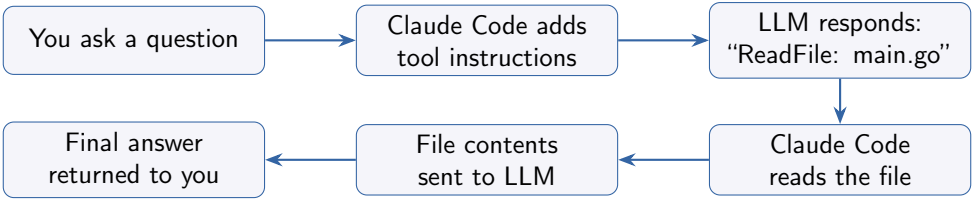
#	Model	Score
1	GPT-5.4	57.7%
2	GPT-5.3-Codex	56.8%
3	GPT-5.2-Codex	56.4%
4	GPT-5.2	55.6%
5	Claude Opus 4.5	45.9%

Contamination caveat

OpenAI found training data leakage in Verified. Pro is cleaner but fewer models submit.

Source: swebench.com, Scale Labs (March 2026). Scores change rapidly — check leaderboards for latest.

The Tool Use Mechanism



Why Claude's tool use matters

Strong tool use ⇒ tackles harder tasks, extensible platform, better security (no full-codebase indexing).

Setup & Project Initialization

Installation:

```
npm install -g @anthropic-ai/claude-code
cd your-project
claude # launches Claude Code
```

The `/init` command:

- Analyzes your entire codebase
- Creates a `CLAUDE.md` summary (architecture, commands, patterns)
- Included in *every* request — persistent system prompt

Tip

Press **Shift+Tab** to cycle permission modes (e.g. auto-accept edits for the session).

Permissions

Three tool approval tiers:

Tool type	Example	Approval
Read-only	File reads, Grep	Automatic
Bash commands	Shell execution	Per command
File modification	Edit, Write	Per session

Permission modes (cycle with **Shift+Tab**):

- `default` — ask each tool
- `acceptEdits` — auto-approve edits
- `plan` — read-only
- `auto` — smart auto-approve
- `bypass` — skip all prompts

Pre-approve in `settings.json`:

```
"permissions": { "allow": ["Bash(npm run *)"],
                 "deny": ["Read(.env)"] }
```

Evaluation order: **deny** > ask > allow (deny always wins).

Adding Context: CLAUDE.md

Three locations, three scopes:

File	Scope	Shared?
`CLAUDE.md`	Project	Yes (committed)
`CLAUDE.local.md`	Project	No (personal)
`~/.claude/CLAUDE.md`	All projects	No (global)

Memory mode with #:

```
# Use comments sparingly. Only comment complex code.
```

Claude merges this instruction into your `CLAUDE.md` automatically.

File mentions with @:

```
How does the auth system work? @auth
```

Includes the file's contents in the request. Also works inside `CLAUDE.md`.

Making Changes

Key techniques:

- **Screenshots** — paste images directly; Claude reads error messages, UI issues
- **Planning mode** — one of the `Shift+Tab` modes; Claude outlines steps before executing
- **Adaptive thinking** — Claude adjusts reasoning depth; tune with `/effort`
- **Auto mode** — classifier auto-approves safe operations, blocks risky ones

Workflow

1. Describe the problem or feature
2. Let Claude explore, plan, and implement
3. Review the diff, approve or redirect

Thinking & Effort Levels

Adaptive Thinking (Opus 4.6 / Sonnet 4.6)

- Claude **dynamically allocates** reasoning depth
- Controlled via `/effort` command:

`low`	Fast, cheap — straightforward tasks
`medium`	Default balanced mode
`high`	Deep reasoning — complex problems
`max`	No token limit (Opus 4.6 only)

Magic keyword: include `ultrathink` anywhere in your prompt → sets effort to `high` for that one turn.

Controls

- `/effort high` — session-wide
- `-effort high` — at startup
- `Alt+T` — toggle thinking on/off
- `Ctrl+O` — show thinking output

Common mistake

"think harder" in your prompt does **nothing**. Only `ultrathink` and `/effort` actually change reasoning depth.

Controlling Context

Interrupt & Redirect

- **Escape** — stop mid-response, redirect
- **Escape** $\times$ **2** — rewind to earlier point
- Combine with # to fix repeating errors

Commands

- `/compact` — summarize history, keep knowledge
- `/clear` — fresh start, no prior context
- Auto-compaction at $\sim$95% context capacity

When to use what

`/compact` when switching to a *related* task (preserves context).
`/clear` when switching to an *unrelated* task (avoids confusion).

Custom Commands

Create reusable commands:

1. Create `.claude/commands/audit.md`
2. Filename becomes the command: `/audit`
3. Restart Claude Code

Commands with arguments (`$ARGUMENTS`):

```
Write comprehensive tests for: $ARGUMENTS

Testing conventions:
* Use Vitest with React Testing Library
* Place test files in __tests__/ next to source
* Name test files as [filename].test.ts(x)

Coverage: happy paths, edge cases, error states
```

Usage:

```
/write_tests the use-auth.ts file in the hooks directory
```

Handy Commands

Side questions

- `/btw <question>` — ask without polluting history
- Uses parent context, no tool access
- Single response, then dismissed
- Low cost (reuses prompt cache)

File references

- `@filename` — include file in context
- `@file.py#5-10` — specific line range
- `@folder/` — directory listing
- `Alt+K` (VS Code) — auto-insert selection

Diagnostics & Info

- `/doctor` — diagnose installation issues
- `/context` — visualize context usage
- `/cost` — token usage statistics
- `/diff` — interactive diff viewer

Session management

- `claude -n "name"` — name a session
- `claude -continue` — resume latest
- `/branch` — fork conversation
- `/export` — save as text file

MCP Servers

Model Context Protocol — extend Claude with new tools.

Example: Playwright (browser control)

```
claude mcp add playwright npx @playwright/mcp@latest
```

Pre-approve permissions in `.claude/settings.local.json`:

```
{
  "permissions": {
    "allow": ["mcp__playwright"],
    "deny": []
  }
}
```

Ecosystem: database interactions, API testing, cloud integrations, file operations, dev tool automation...

Deferred loading: MCP tools lazy-load via `ToolSearch`, keeping unused tool schemas out of context.

MCP in Practice: Context7

Problem: Claude's training data lags behind library releases → hallucinated APIs.
Solution: **Context7** MCP fetches *current* official documentation on demand.

Setup (one command):

```
claude mcp add context7 -- npx -y @upstash/context7-mcp@latest
```

Usage — just say "use context7":

```
> How does Next.js after() work? use context7
```

How it works:

1. `resolve-library-id` → finds the library
2. `query-docs` → fetches version-specific docs
3. Claude answers with *current* API

Why it matters:

- No hallucinated deprecated methods
- Working code on first try
- Version-specific examples
- Free, open-source (Upstash)

GitHub Integration

Run `/install-github-app` in Claude Code.

Two workflows added to `.github/workflows/`:

1. **Mention Action** — `@claude` in issues/PRs → Claude analyzes & responds
2. **PR Review Action** — auto-reviews every pull request

Customization options:

- `custom_instructions:` — project-specific context
- `mcp_config:` — attach MCP servers (e.g. Playwright)
- `allowed_tools:` — explicit tool permissions (required in CI)

Hooks: Architecture

PreToolUse

- Runs *before* tool call
- Can **block or rewrite** the operation
- Access control, validation

PostToolUse

- Runs *after* tool call
- Cannot block (already done)
- Formatting, testing, logging

Hook types: `"command"` (shell) | `"http"` (POST JSON to URL)
Config locations: `~/.claude/settings.json` (global) | `.claude/settings.json` (team) | `.claude/settings.local.json` (personal)

Hook Configuration

PreToolUse example — guard specific tool calls:

```
"PreToolUse": [{
  "matcher": "Read|Grep",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/read_hook.js"
  }]
}]
```

PostToolUse example — run formatter after edits:

```
"PostToolUse": [{
  "matcher": "Write|Edit|MultiEdit",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/format_hook.js"
  }]
}]
```

Implementing a Hook: Protect `.env`

```
async function main() {
  const chunks = [];
  for await (const chunk of process.stdin) {
    chunks.push(chunk);
  }
  const toolArgs = JSON.parse(
    Buffer.concat(chunks).toString()
  );

  const readPath =
    toolArgs.tool_input?.file_path
    || toolArgs.tool_input?.path || "";

  if (readPath.includes('.env')) {
    console.error("You cannot read the .env file");
    process.exit(2); // exit code 2 = block
  }
}

main();
```

Hook receives tool call data via `stdin` as JSON. Exit code 2 blocks the operation with an error message back to Claude.

Hook Gotchas & Practical Recipes

Common Gotchas

- Use **absolute paths** in hook commands
- Restart Claude Code after changing hooks
- Test hooks with simple cases first

Useful hook ideas:

- **TypeScript type-checking** — run `tsc` after edits, feed errors back
- **Auto-formatting** — run Prettier/Black after writes
- **Query duplication prevention** — detect duplicate SQL/API calls
- **Test runner** — auto-run tests on changed files
- **Sensitive file protection** — block access to credentials

Other hook events: Notification, Stop, SubagentStop, PreCompact, UserPromptSubmit, SessionStart, SessionEnd.

The Claude Code SDK

Claude Agent SDK — run Claude Code programmatically (TypeScript, Python, CLI).

```
import { query } from "@anthropic-ai/claude-code";

const prompt = "Look for duplicate queries in ./src/queries";

for await (const message of query({ prompt })) {
  console.log(JSON.stringify(message, null, 2));
}
```

Permissions: read-only by default. Enable writes with:

```
for await (const message of query({
  prompt,
  options: { allowedTools: ["Edit"] }
})) { /* ... */ }
```

Use cases: Git hooks for code review, CI/CD quality checks, automated documentation, build optimization scripts.

Workflow Tips

1. Plan before you code

- Use **plan mode** (`Shift+Tab`) for unfamiliar code
- Ask Claude to research first: "google how X works, then implement"
- For big features: have Claude interview you, write a spec, then start fresh

2. Keep memory fresh

- Periodically: "update your memory with what you learned"
- `/memory` to browse and prune stale entries
- Keep `CLAUDE.md` under 200 lines — specificity > length

3. Ask Claude to improve your workflow

- "What workflow improvements would you suggest for this project?"
- "Review my CLAUDE.md — what's redundant or missing?"
- `/insights` for session analytics

4. Manage context aggressively

- `/clear` between unrelated tasks
- `/compact` when switching related tasks
- Use subagents for long explorations
- `/btw` for quick side questions

Common Pitfalls

Kitchen sink sessions

Mixing unrelated tasks fills context with noise.
Fix: `/clear` between unrelated tasks.

Correcting the same bug 3×

Failed attempts accumulate, confusing Claude.
Fix: After 2 fails, `/clear` and write a better prompt.

Vague prompts

"fix the login bug" gives Claude nothing to work with.

Good habits

- Provide **verification**: tests, screenshots, expected output
- Use `@file#5-10` to point at specific code
- Name sessions: `claude -n "feature-auth"`
- Start in plan mode for unfamiliar codebases
- Paste images of errors — Claude reads screenshots
- Scope exploration: "investigate token refresh in `src/auth/` only"

Summary

Core Concepts

- Tool use makes LLMs practical
- `CLAUDE.md` = persistent context
- `@` mentions for file focus
- `/compact` vs `/clear`

Power Features

- Custom commands (`$ARGUMENTS`)
- MCP servers (deferred tool loading)
- Hooks (block, rewrite, HTTP)
- Claude Agent SDK
- Auto mode, adaptive thinking
- `/model` to switch models mid-session
- Voice (`/voice`), remote control

Questions?

Resources & Further Learning

Official Documentation

- **Claude Code Docs** — features, platforms, guides
- `github.com/anthropics/claude-code`
- `github.com/anthropics/skills` — official skills

Learning

- **Anthropic Academy** — free 21-lesson course
- **Coursera** — same course, with certificate

Standards

- `agentskills.io` — Agent Skills open standard
- `swebench.com` — coding benchmark

Community Collections

- **Awesome Claude Code Toolkit**
  135 agents, 35+ skills, 150+ plugins
- **Awesome Claude Skills**
  Domain expertise & workflows

Deep Dives

- **Advanced Tool Use** (Anthropic Engineering)
- **API Docs** — models, tool use, vision